

m-numerik

Lukas Arnold	Simone Arnold	Florian Bagemihl
Matthias Baitsch	Marc Fehr	Maik Poetzsch
	Sebastian Seipel	

2026-09-17

Table of contents

Preamble	3
Introduction	4
1 Numerical Methods	5
2 Integration	6
3 Differentiation	22

Preamble



Modules for Computer-Assisted Data Analysis. “Numerics” by Lukas Arnold, Simone Arnold, Florian Bagemihl, Matthias Baitsch, Marc Fehr, Maik Poetzsch, and Sebastian Seipel is licensed under [CC BY 4.0](https://creativecommons.org/licenses/by/4.0/). The work is available at <https://github.com/bausteine-der-datenanalyse/m-numerik-en>. All logos and content marked otherwise are excluded from the license. 2026

Citation Suggestion

Arnold, Lukas, Simone Arnold, Matthias Baitsch, Marc Fehr, Maik Poetzsch, and Sebastian Seipel. 2024. “Modules for Computer-Assisted Data Analysis. Numerics Module.” <https://github.com/bausteine-der-datenanalyse/m-numerik-en>.

BibTeX Template

```
@misc{BCD-Styleguide-2024,  
  title={Modules for Computer-Assisted Data Analysis. Numerics Module},  
  author={Arnold, Lukas and Arnold, Simone and Baitsch, Matthias and Fehr, Marc and Poetzsch, Maik},  
  year={2024},  
  url={https://github.com/bausteine-der-datenanalyse/m-numerik-en}}
```

Introduction

Prerequisites

- Python Basics
- Importing Additional Packages
- Plotting with Matplotlib

Used Packages and Datasets

Packages

- [NumPy](#)
- [Matplotlib](#)

Datasets

Estimated Completion Time

Estimated completion time: 2h

Learning Objectives

- What is Numerical Computing
- Implementing Numerical Methods
- Numerical Integration
- Numerical Differentiation

1 Numerical Methods

[Numerical methods](#) are a branch of mathematics that focuses on techniques for finding solutions or approximate solutions to problems that cannot be solved analytically.

This includes areas such as integration and differentiation, as well as differential equations. Many engineering simulation programs use numerical methods to compute complex scenarios. In particular, solving ordinary and partial differential equations would be nearly impossible without numerical methods.

Applications include, for example:

- Deformation of bodies under applied forces
- Vibrations of buildings
- Temporal evolution of substance concentrations
- Fluid dynamics: hydraulic engineering, flow around buildings, ventilation, smoke dispersion
- Heat transfer in components

2 Integration

The computation of integrals is used, for example, in determining areas or total forces. Formally, the definite integral I of the function $f(x)$ over the interval $x \in [a, b]$ is represented as follows:

$$I = \int_a^b f(x) \, dx$$

In general, the integral cannot be solved analytically, as the antiderivative $F(x)$ is not easy to determine. In such cases, numerical methods can be used to approximate the integral value. Numerical integration is often also referred to as numerical quadrature.

This chapter provides a brief overview of numerical integration methods:

- Upper and lower sums
- Quadrature
- Monte Carlo

2.0.1 Upper and Lower Sum

```
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
```

One of the most fundamental ways to compute integrals of functions is through the [upper and lower sums](#). They approximate the integral by bounding it from above and below. As the number of sample points, i.e., positions where the function is evaluated, increases, both bounds converge to the true integral value.

2.1 Definition

To construct the upper and lower sums, evenly spaced sample points on the interval $[a, b]$ are required. If $n + 1$ sample points are chosen, we have:

$$a = x_0 < x_1 < \dots < x_n = b$$

The spacing between the points is $\Delta x = (b - a)/(n - 1)$. On each of the n subintervals $[x_{i-1}, x_i]$, the maximum and minimum values of the function $f(x)$ are determined and defined as O_i and U_i , respectively.

$$O_i = \max (f(x)|x \in [x_{i-1}, x_i])$$

$$U_i = \min (f(x)|x \in [x_{i-1}, x_i])$$

The desired approximation of the integral is the sum of O_i and U_i multiplied by the width of the subinterval, Δx :

$$\sum_{i=1}^n \Delta x U_i \lesssim I \lesssim \sum_{i=1}^n \Delta x O_i$$

2.2 Example

As an example, consider the following integral:

$$I = \int_0^2 \sin(3x) + 2x \, dx$$

```
def func(x):
    return np.sin(3*x) + 2*x

# Data for visualization
x = np.linspace(0, 2, 100)
y = func(x)

# Exact solution
I_exact = (-1/3*np.cos(3*2) + 2**2) - (-1/3)
```

First, the support points are evenly distributed in the interval $[0, 2]$.

```
n = 5

xi = np.linspace(0, 2, n)
yi = func(xi)
```

Both sums require the extreme values of the function to be integrated within the subintervals. These are determined by evaluating the function on the subinterval. For the following visualization, the set of sums also has n elements.

```
upper = np.zeros(n)
lower = np.zeros(n)

for i in range(len(upper)-1):
    cx = np.linspace(xi[i], xi[i+1], 50)
    cy = func(cx)
    upper[i+1] = np.max(cy)
    lower[i+1] = np.min(cy)
```

The first elements of the two cumulative lists are set to the first function values, this is only for the subsequent display.

```
upper[0] = yi[0]
lower[0] = yi[0]
```

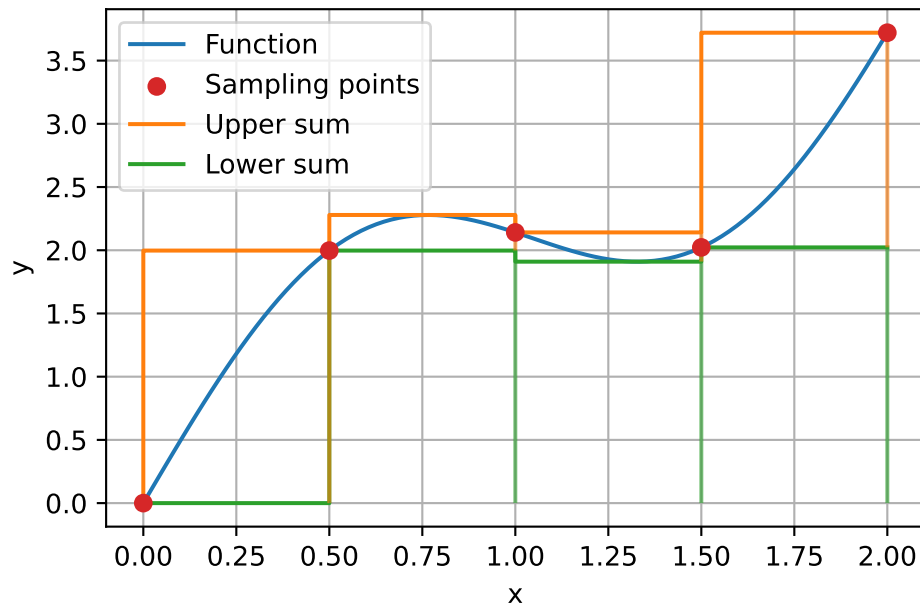
Visualization of individual functions

```
plt.plot(x, y, label='Function')
plt.scatter(xi, yi, label='Sampling points', c='C3', zorder=3)
plt.plot(xi, upper, drawstyle='steps-pre', label='Upper sum')
plt.plot(xi, lower, drawstyle='steps-pre', label='Lower sum')

plt.vlines(xi, ymin=lower, ymax=upper, color='C1', alpha=0.6)
plt.vlines(xi, ymin=0, ymax=lower, color='C2', alpha=0.6)

plt.xlabel('x')
plt.ylabel('y')

plt.grid()
plt.legend();
```



The above procedure can now be summarized in a function that returns the sums of the two sequences.

```
def up_down_sum(n, start=0, end=2):
    x_vals = np.linspace(start, end, n)
    y_vals = func(x_vals)
    dx = x_vals[1] - x_vals[0]

    sum_upper = 0
    sum_lower = 0

    for i in range(n-1):
        sub_x = np.linspace(x_vals[i], x_vals[i+1], 50)
        sub_y = func(sub_x)
        upper = np.max(sub_y)
        lower = np.min(sub_y)
        sum_upper += dx * upper
        sum_lower += dx * lower

    return sum_upper, sum_lower
```

For a systematic study of convergence behavior, the integration function is called for different numbers of nodes.

```

n_max = 100
ns = np.arange(2, n_max, 1, dtype=int)
os = np.zeros(len(ns))
us = np.zeros(len(ns))

for i, n in enumerate(ns):
    o, u = up_down_sum(n)
    os[i] = o
    us[i] = u

```

The graphical representation of the two sums shows a continuous convergence of these.

```

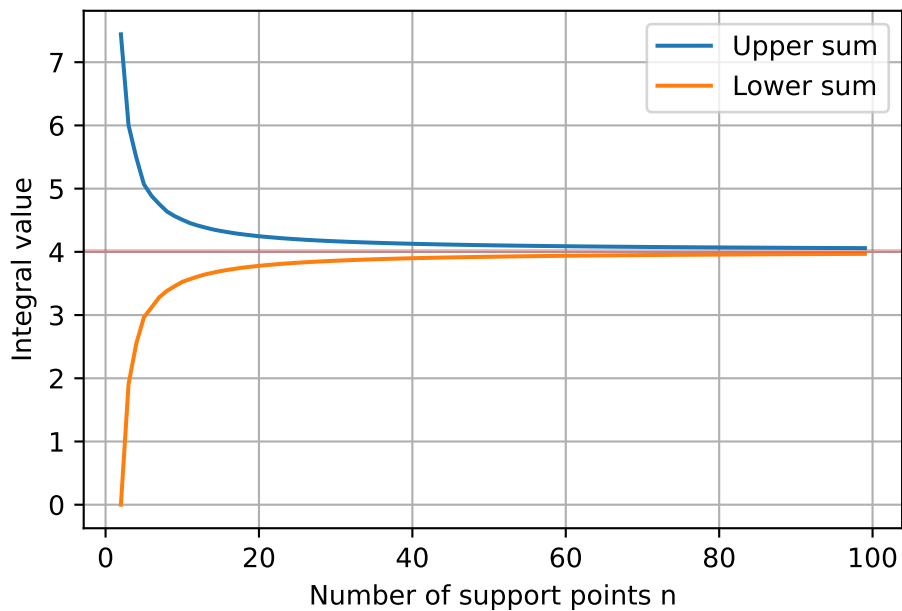
plt.plot(ns, os, label='Upper sum')
plt.plot(ns, us, label='Lower sum')

plt.axhline(y=I_exact, color='C3', alpha=0.3)

plt.xlabel('Number of support points n')
plt.ylabel('Integral value')

plt.grid()
plt.legend();

```



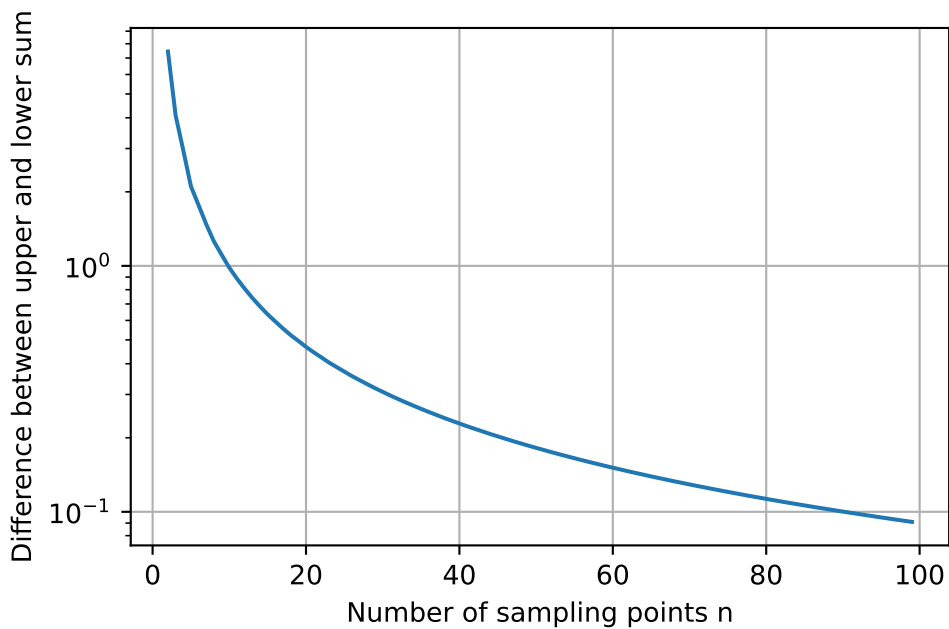
This becomes particularly evident when the difference between the two sums is plotted. With a logarithmic representation, the continuous convergence can also be read quantitatively.

```
plt.plot(ns, os-us)

plt.xlabel('Number of sampling points n')
plt.ylabel('Difference between upper and lower sum')

# plt.xscale('log')
plt.yscale('log')

plt.grid();
```



2.3 Interpolation

In the formation of the upper and lower sums, the function to be integrated was approximated by a constant value in the subintervals between the support points. A more accurate calculation of the integral can be achieved through better interpolation. Polynomials are suitable for this because they are easy to integrate.

2.3.1 Trapezoidal Rule

The trapezoidal rule is based on approximating the function to be integrated by straight lines, i.e., polynomials of degree 1, over the subintervals. The approximation of the integral value results accordingly from the areas of the resulting trapezoids.

As in the previous chapter, the method is demonstrated using the following function:

$$I = \int_0^2 \sin(3x) + 2x \, dx$$

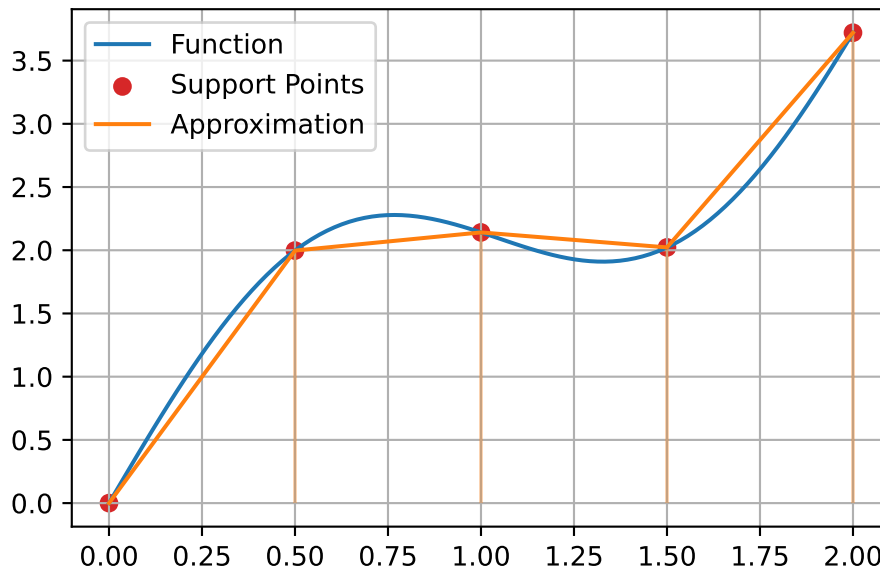
```
def func(x):  
    return np.sin(3*x) + 2*x  
  
# Data for visualization  
x = np.linspace(0, 2, 100)  
y = func(x)  
  
# Exact solution  
I_exact = (-1/3*np.cos(3*2) + 2**2) - (-1/3)
```

Generation of support points

```
n = 5  
  
xi = np.linspace(0, 2, n)  
yi = func(xi)
```

First, we visualize the method.

```
plt.plot(x, y, label='Function')  
plt.scatter(xi, yi, label='Support Points', c='C3')  
plt.plot(xi, yi, label='Approximation', c='C1')  
  
plt.vlines(xi, ymin=0, ymax=yi, color='C1', alpha=0.3)  
  
plt.grid()  
plt.legend();
```



The integration itself can be performed using the `function` `scipy.integrate.trapezoid`.

```
res = scipy.integrate.trapezoid(yi, xi)
print(f"Integral value with {n} sample points: {res:.4f}")
```

Integral value with 5 sample points: 4.0107

The computed value approaches the exact value as the number of support points increases.

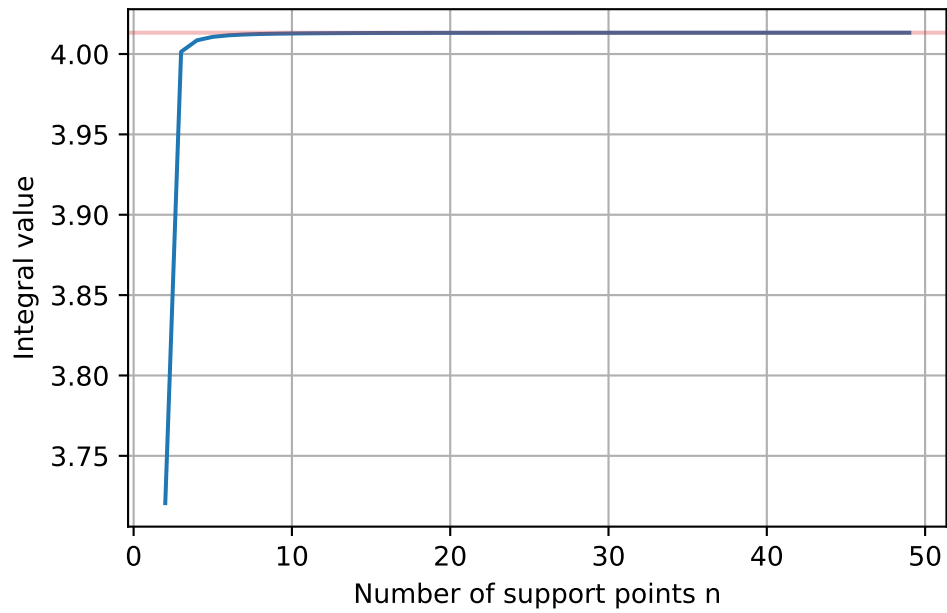
```
n_max = 50
ns = np.arange(2, n_max, 1, dtype=int)
tr = np.zeros(len(ns))

for i, n in enumerate(ns):
    xi = np.linspace(0, 2, n)
    yi = func(xi)
    tr[i] = scipy.integrate.trapezoid(yi, xi)
```

```
plt.plot(ns, tr)
plt.axhline(y=I_exact, color='C3', alpha=0.3)

plt.xlabel('Number of support points n')
plt.ylabel('Integral value')

plt.grid();
```

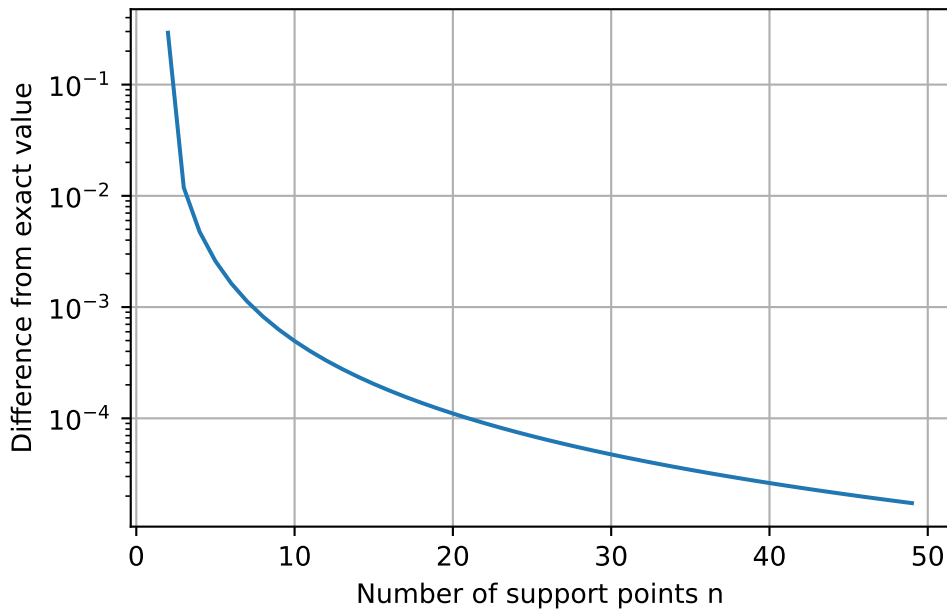


```
plt.plot(ns, np.abs(tr-I_exact))

plt.xlabel('Number of support points n')
plt.ylabel('Difference from exact value')

# plt.xscale('log')
plt.yscale('log')

plt.grid();
```



2.3.2 Simpson's Rule

Using a second-degree polynomial leads to Simpson's rule.

Here, the function is evaluated at a midpoint within each subinterval, and this value, together with the values at the endpoints, is used to determine the polynomial coefficients.

The example above visually demonstrates Simpson's rule.

```
n = 5

xi = np.linspace(0, 2, n)
yi = func(xi)

plt.plot(x, y, label='Function')
plt.scatter(xi, yi, label='Data Points', c='C3')

# Compute and plot the polynomials
for i in range(n-1):
    dx = xi[i+1] - xi[i]
    cx = (xi[i] + xi[i+1]) / 2
    cy = func(cx)
```

```

P = np.polyfit([xi[i], cx, xi[i+1]], [yi[i], cy, yi[i+1]], 2)

Px = np.linspace(xi[i], xi[i+1], 20)
Py = np.polyval(P, Px)

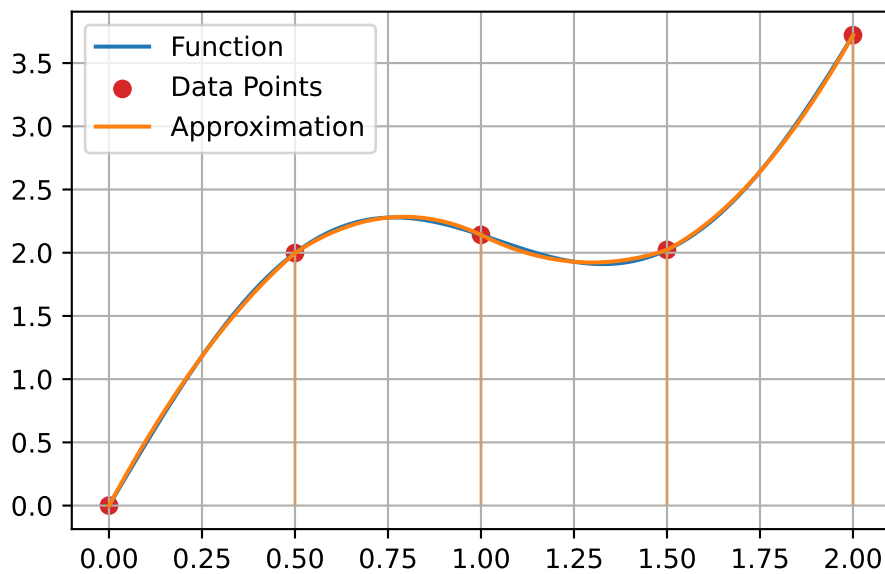
label=None
if i==0:
    label='Approximation'

plt.plot(Px, Py, color='C1', label=label)

plt.vlines(xi, ymin=0, ymax=yi, color='C1', alpha=0.3)

plt.grid()
plt.legend();

```



The Simpson's rule is already implemented in [function `scipy.integrate.simpson`](#). Below, only the difference to the trapezoidal rule is demonstrated.

```

n_max = 50
ns = np.arange(3, n_max, 2, dtype=int)
si = np.zeros(len(ns))
tr = np.zeros(len(ns))

for i, n in enumerate(ns):

```

```

xi = np.linspace(0, 2, n)
yi = func(xi)
si[i] = scipy.integrate.simpson(yi, xi)
tr[i] = scipy.integrate.trapezoid(yi, xi)

```

```

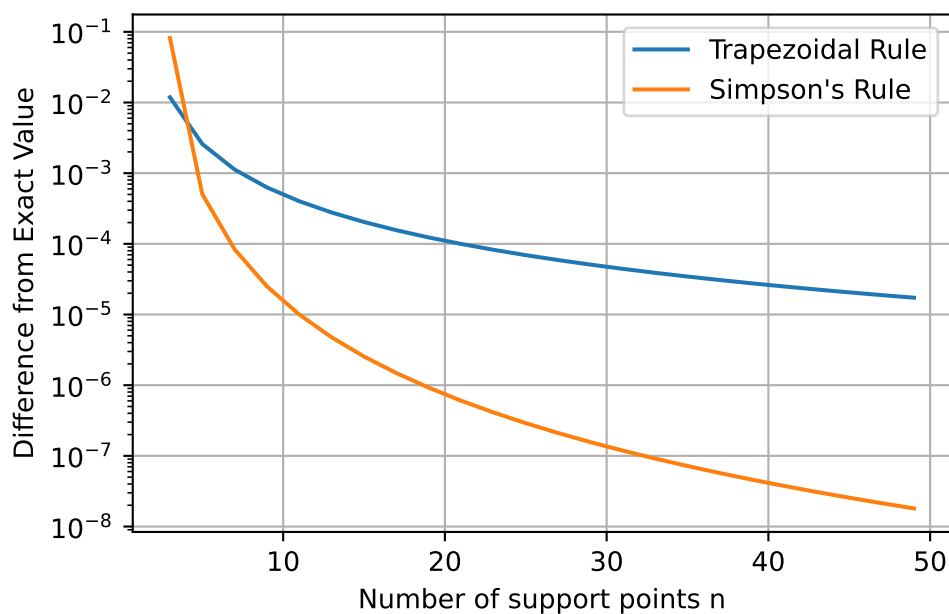
plt.plot(ns, np.abs(tr-I_exact), label='Trapezoidal Rule')
plt.plot(ns, np.abs(si-I_exact), label='Simpson\'s Rule')

plt.xlabel('Number of support points n')
plt.ylabel('Difference from Exact Value')

# plt.xscale('log')
plt.yscale('log')

plt.legend()
plt.grid();

```



2.4 Monte Carlo

A completely different approach to integration is followed with the [Monte Carlo method](#). Here, random points x_i are generated within the desired integration range. The mean of the

corresponding sum of function values $f(x_i)$ approximates the integral. Especially for a small number of random values, the result can deviate significantly from the exact value. The advantage of the method becomes apparent for high-dimensional integrals.

For $n \gg 1$ random sample points $x_i \in [a, b]$, the following approximation holds:

$$I = \int_a^b f(x) \, dx \approx \frac{b-a}{n} \sum_{i=1}^n f(x_i)$$

For the example from the previous chapters, we have

```
def func(x):
    return np.sin(3*x) + 2*x

# Data for visualization
x = np.linspace(0, 2, 100)
y = func(x)

# Exact solution
I_exact = (-1/3*np.cos(3*2) + 2**2) - (-1/3)

n = 2000
xi = np.random.random(n) * 2
yi = func(xi)
I = 2 * 1/n * np.sum(yi)
print(f"Integral value for {n} sample points: {I:.4f}")
```

Integral value for 2000 sample points: 4.0264

```
n_max = 50000
dn = 250
ns = np.arange(dn, n_max, dn, dtype=int)
mc = np.zeros(len(ns))

xi = np.zeros(n_max)

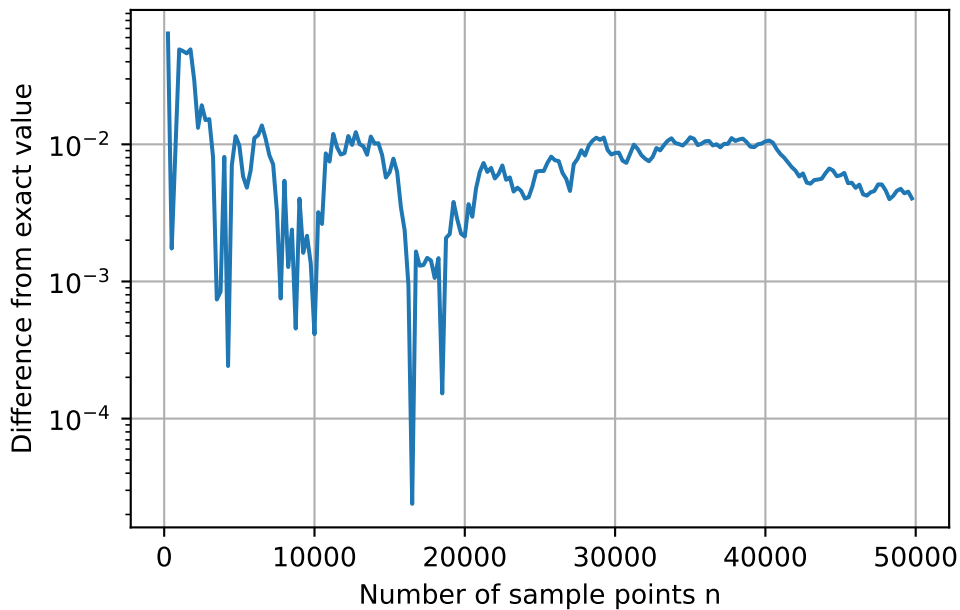
for i, n in enumerate(ns):
    xi[n-dn:n] = np.random.random(dn) * 2
    yi = func(xi[:n])
    mc[i] = 2 * 1/n * np.sum(yi)
```

```
plt.plot(ns, np.abs(mc - I_exact))

plt.xlabel('Number of sample points n')
plt.ylabel('Difference from exact value')

# plt.xscale('log')
plt.yscale('log')

plt.grid();
```



Alternatively, the area ratio between the function to be integrated and a reference area A_r can be used. For this purpose, n pairs of random numbers (x_i, y_i) are generated and counted to see how many of them lie within the target area. The assumption is that both ratios converge for large n . In the simplest case, when $f(x) \geq 0$, the following estimate applies:

$$I \approx \frac{A_r \cdot |\{y_i \mid y_i < f(x_i)\}|}{n}$$

In the example above, the area $[0, 2] \times [0, 4] = 8$ can be used as the reference area.

```
n = 2000
xi = np.random.random(n) * 2 # x-coordinates of random points
yi = np.random.random(n) * 4 # y-coordinates of random points
```

```

z = np.sum(yi < func(xi)) # count points under the function

I = z / n * 8 # approximate integral
print(f"Integral value with {n} sample points: {I}")

```

Integral value with 2000 sample points: 3.844

```

max_n = 50000
step_n = 250
n_values = np.arange(step_n, max_n, step_n, dtype=int)
mc_estimates = np.zeros(len(n_values))

x_vals = np.zeros(max_n)
y_vals = np.zeros(max_n)

for i, n in enumerate(n_values):
    x_vals[n-step_n:n] = np.random.random(step_n) * 2
    y_vals[n-step_n:n] = np.random.random(step_n) * 4
    count = np.sum(y_vals[:n] < func(x_vals[:n]))
    mc_estimates[i] = count / n * 8

```

```

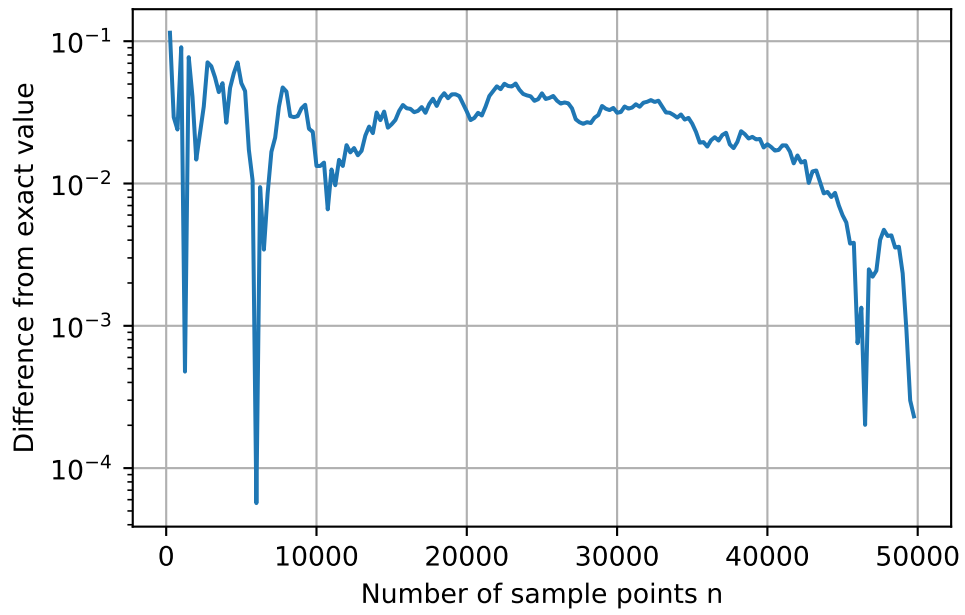
plt.plot(n_values, np.abs(mc_estimates-I_exact))

plt.xlabel('Number of sample points n')
plt.ylabel('Difference from exact value')

# plt.xscale('log')
plt.yscale('log')

plt.grid();

```



3 Differentiation

The numerical calculation of derivatives is demonstrated here using two approaches: first, as a difference quotient, and second, through polynomial fitting. These methods are applied, for example, when searching for extrema in experimental or simulation data, solving differential equations, or performing optimization procedures.

Although analytically computing a derivative is often much easier than integration, it is not directly possible in the cases mentioned above. The goal is always to find the derivative $f'(x)$ of a function $f(x)$ or of a discrete set of points (x_i, y_i) at a specific location $x = x_0$ or over an interval.

The basic idea behind the difference quotients or difference formulas presented here is to approximate the function to be differentiated using a Taylor expansion at multiple points. This allows solving for the desired derivative at the corresponding expansion point.

3.1 Taylor Expansion

Using the [Taylor expansion](#), any function $f(x)$ that is continuously differentiable any number of times can be approximated arbitrarily closely around an expansion point x_0 . The functional dependence is now considered with respect to the variable h , which is only examined in the immediate vicinity of x_0 . The Taylor expansion is given by:

$$\begin{aligned} f(x_0 + h) &= \sum_{i=0}^{\infty} \frac{1}{i!} f^{(i)}(x_0) \cdot h^i \\ &= f(x_0) + f'(x_0) \cdot h + \frac{1}{2} f''(x_0) \cdot h^2 + \frac{1}{6} f'''(x_0) \cdot h^3 + \dots \end{aligned}$$

This expansion can also be considered only up to a given order. For example, the expansion up to order $\mathcal{O}(h^3)$ takes the form:

$$f(x_0 + h) = f(x_0) + f'(x_0) \cdot h + \frac{1}{2} f''(x_0) \cdot h^2 + \mathcal{O}(h^3)$$

Here, the Landau symbol $\mathcal{O}$ indicates the order, summarizing the neglected terms, here starting from h^3 , as an approximation error. The order specifies how quickly or with which functional relationship the approximation error approaches zero as $h \rightarrow 0$.

A graphical representation of the first elements of the series further illustrates the basic idea. The following example expands the function

$$f(x) = \sin(3x) + 2x$$

around the point $x_0 = 0.85$.

```
def func(x, p=0):
    if p==0:
        return np.sin(3*x) + 2*x
    if p==1:
        return 3*np.cos(3*x) + 2
    if p==2:
        return -9*np.sin(3*x)
    if p==3:
        return -27*np.cos(3*x)
    return None
```

```
# Data for visualization
x = np.linspace(0, 2, 100)
y = func(x, p=0)
```

```
x0 = 0.85
```

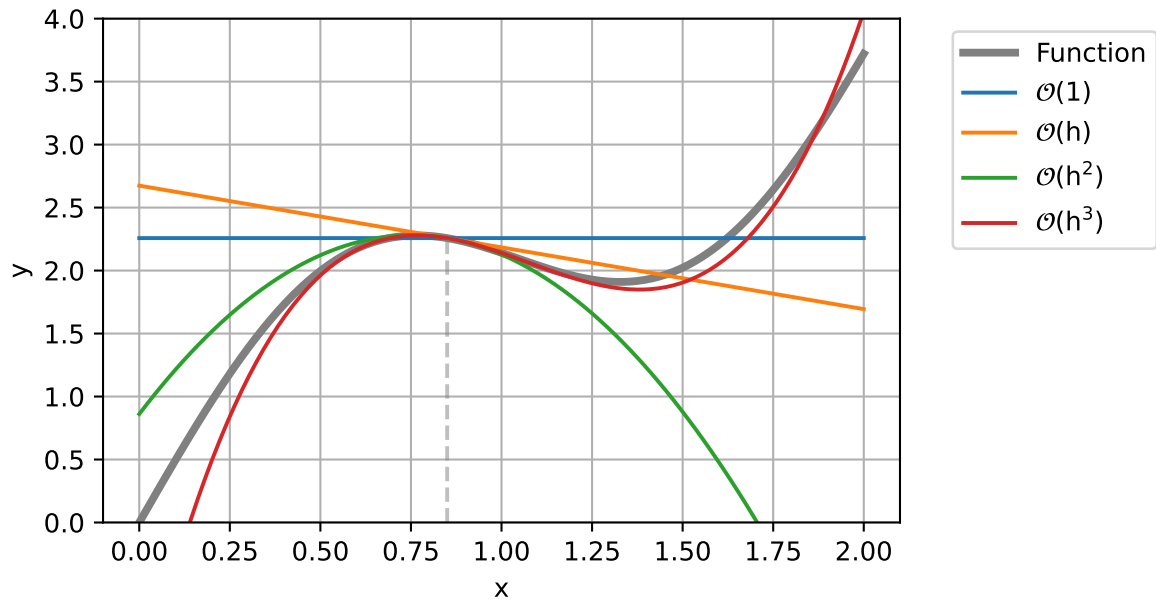
```
# Taylor elements
te = []
te.append(0*(x-x0) + func(x0, p=0))
te.append((x-x0) * func(x0, p=1))
te.append((x-x0)**2 * func(x0, p=2) * 1/2)
te.append((x-x0)**3 * func(x0, p=3) * 1/6)
```

```
plt.plot(x, y, color='Grey', lw=3, label="Function")
plt.plot(x, te[0], label=r"$\mathsf{\mathcal{0}(1)}$")
plt.plot(x, te[0] + te[1], label=r"$\mathsf{\mathcal{0}(h)}$")
plt.plot(x, te[0] + te[1] + te[2], label=r"$\mathsf{\mathcal{0}(h^2)}$")
plt.plot(x, te[0] + te[1] + te[2] + te[3], label=r"$\mathsf{\mathcal{0}(h^3)}$")

plt.vlines(x0, ymin=0, ymax=func(x0), color='Grey', ls='--', alpha=0.5)

plt.ylim([0,4])
```

```
plt.legend(bbox_to_anchor=(1.05, 1.0), loc='upper left')
plt.grid()
plt.xlabel('x')
plt.ylabel('y');
```



3.2 Difference Formulas

In this section, formulas for approximating derivatives by forming differences of function values are introduced. These formulas are all based on the Taylor expansion and can be formulated for arbitrary derivatives and orders. The simplest ones are presented here.

3.2.1 First Derivative, First Order

The simplest difference formula is obtained from the Taylor series up to $\mathcal{O}(h^2)$. Here, the series can be directly rearranged for the desired derivative at the point x_0 .

$$f(x_0 + h) = f(x_0) + f'(x_0)h + \mathcal{O}(h^2)$$

$$\Rightarrow f'(x_0) = \frac{f(x_0 + h) - f(x_0)}{h} + \mathcal{O}(h)$$

This is the forward difference formula of first order for the first derivative. First order means that in the limit $h \rightarrow 0$ the approximation error decreases linearly with the step size.

According to this formula, the function to be differentiated must be evaluated at two points $f(x_0)$ and $f(x_0 + h)$ in order to determine the derivative numerically. In the limit of an arbitrarily small step size, i.e. $h \rightarrow 0$, this quotient approaches the exact derivative at the point x_0 .

The following example demonstrates the approximation using the function

$$f(x) = \sin(3x) + 2x$$

The derivative is approximated at the point $x_0 = 0.85$.

```
def func(x):
    return np.sin(3*x) + 2*x

# Data for visualization
x = np.linspace(0, 2, 100)
y = func(x)

# Exact solution at x=0.85
fp_exactly = 3*np.cos(3*0.85) + 2

# Expansion point and step size
h = 0.25
x0 = 0.85

# Evaluation at the two points
f0 = func(x0)
fh = func(x0 + h)

# Determination of the derivative approximation
fp = (fh - f0) / h

print(f"The numerical approximation of the derivative at the point {x0:.2f}:")
print(f"Approximation with step size {h:.2f}: {fp:.2f}")
print(f"Exact value: {fp_exactly:.2f}")
```

The numerical approximation of the derivative at the point 0.85:
Approximation with step size 0.25: -0.86
Exact value: -0.49

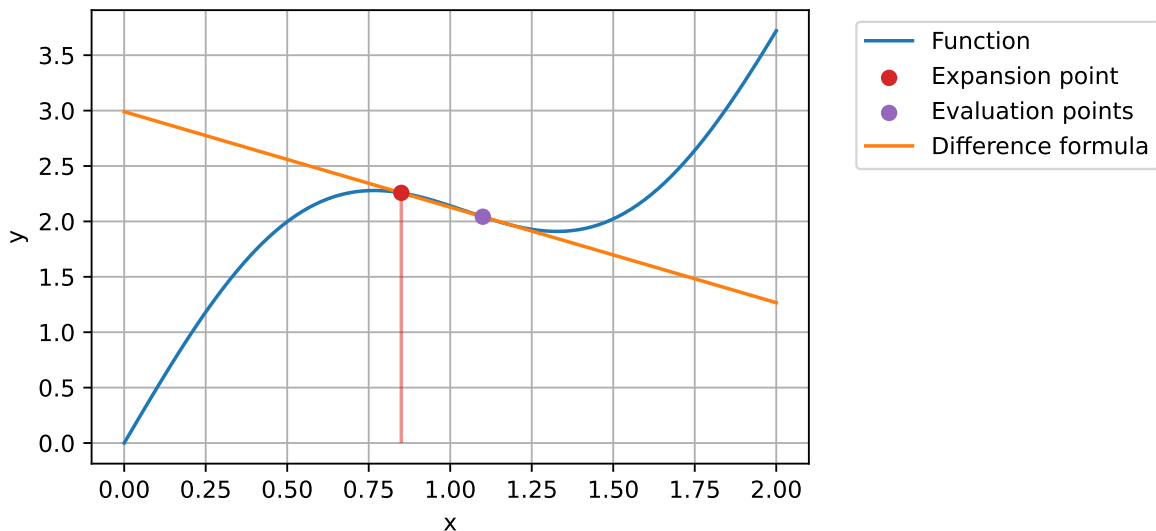
The method can also be illustrated graphically. The desired slope is the slope of the straight line shown in the plot.

```
plt.plot(x, y, label="Function")
plt.scatter([x0], [f0], color='C3', label='Expansion point', zorder=3)
plt.scatter([x0+h], [fh], color='C4', label='Evaluation points', zorder=3)

plt.vlines(x0, ymin=0, ymax=f0, color='C3', alpha=0.5)

plt.plot(x, f0 + fp*(x-x0), label='Difference formula')

plt.xlabel('x')
plt.ylabel('y')
plt.grid()
plt.legend(bbox_to_anchor=(1.05, 1.0), loc='upper left');
```



3.2.2 Second-Order First Derivative

Using the same approach, a second-order difference formula can also be derived. For this, the function is approximated at the points $x_0 - h$ and $x_0 + h$ using the Taylor series up to order $\mathcal{O}(h^3)$.

$$f(x_0 + h) = f(x_0) + f'(x_0) \cdot h + \frac{1}{2}f''(x_0) \cdot h^2 + \mathcal{O}(h^3)$$

$$f(x_0 - h) = f(x_0) - f'(x_0) \cdot h + \frac{1}{2}f''(x_0) \cdot h^2 + \mathcal{O}(h^3)$$

The difference between these two equations gives

$$f(x_0 + h) - f(x_0 - h) = 2f'(x_0) \cdot h + \mathcal{O}(h^3)$$

Rearranging for the derivative at the point x_0 yields

$$f'(x_0) = \frac{f(x_0 + h) - f(x_0 - h)}{2h} + \mathcal{O}(h^2)$$

This is the central difference formula for the first derivative with second-order accuracy. As with the forward formula, the function must be evaluated at two points, but not at the expansion point itself. This symmetry around the expansion point results in improved convergence behavior, which is quadratic in this case.

```
# Evaluate the function at both points
fmh = func(x0 - h)
fph = func(x0 + h)

# Compute the derivative approximation
fp = (fph - fmh) / (2*h)

print(f"The numerical approximation of the derivative at the point {x0:.2f}:")
print(f"Approximation with step size {h:.2f}: {fp:.2f}")
print(f"Exact value: {fp_exactly:.2f}")
```

```
The numerical approximation of the derivative at the point 0.85:
Approximation with step size 0.25: -0.26
Exact value: -0.49
```

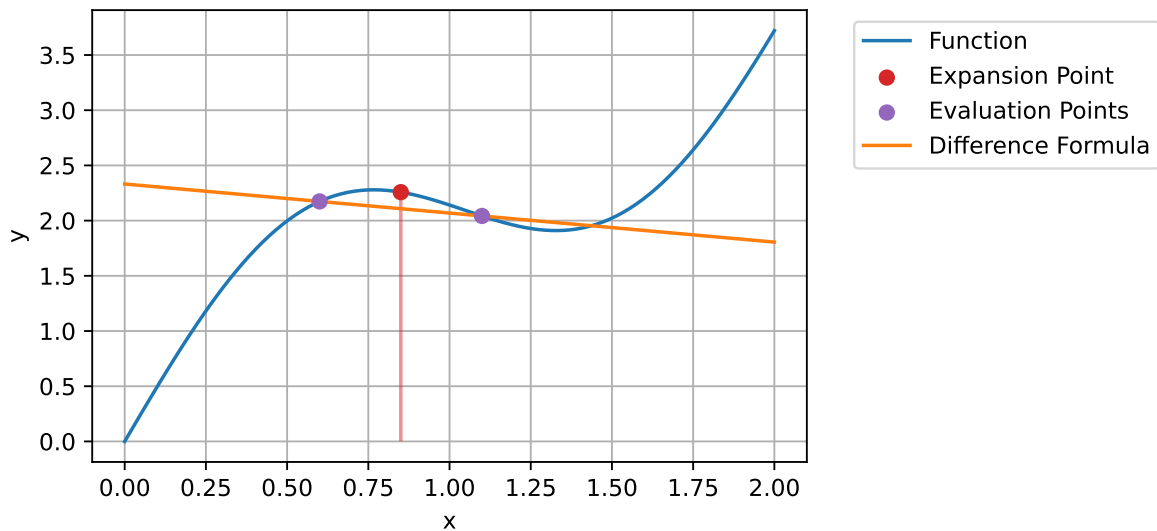
The method can also be represented graphically. The slope sought is the slope of the straight lines drawn.

```
plt.plot(x, y, label="Function")
plt.scatter([x0], [f0], color='C3', label='Expansion Point', zorder=3)
plt.scatter([x0-h, x0+h], [fmh, fph], color='C4', label='Evaluation Points', zorder=3)

plt.vlines(x0, ymin=0, ymax=f0, color='C3', alpha=0.5)
```

```
plt.plot(x, fmh + fp*(x-x0+h), label='Difference Formula')

plt.xlabel('x')
plt.ylabel('y')
plt.grid()
plt.legend(bbox_to_anchor=(1.05, 1.0), loc='upper left');
```



3.3 Second-Order Derivative

Using the same scheme as above, the finite difference formula for the second derivative can also be determined. It is given by

$$f''(x_0) = \frac{f(x_0 - h) - 2f(x_0) + f(x_0 + h)}{h^2} + \mathcal{O}(h^2)$$

3.4 Error Analysis

In this section, approximation errors, i.e., errors arising from difference formulas, and rounding errors, i.e., errors due to the finite precision of digital number representation, are considered.

3.4.1 Approximation Error

The order of the method can be determined by analyzing the error, here with respect to the known exact value. For this purpose, the step size is continuously reduced.

```
def func(x):  
    return np.sin(3*x) + 2*x  
  
# Data for visualization  
x = np.linspace(0, 2, 100)  
y = func(x)  
  
# Exact solution at x=1  
fp_exact = 3*np.cos(3*0.85) + 2
```

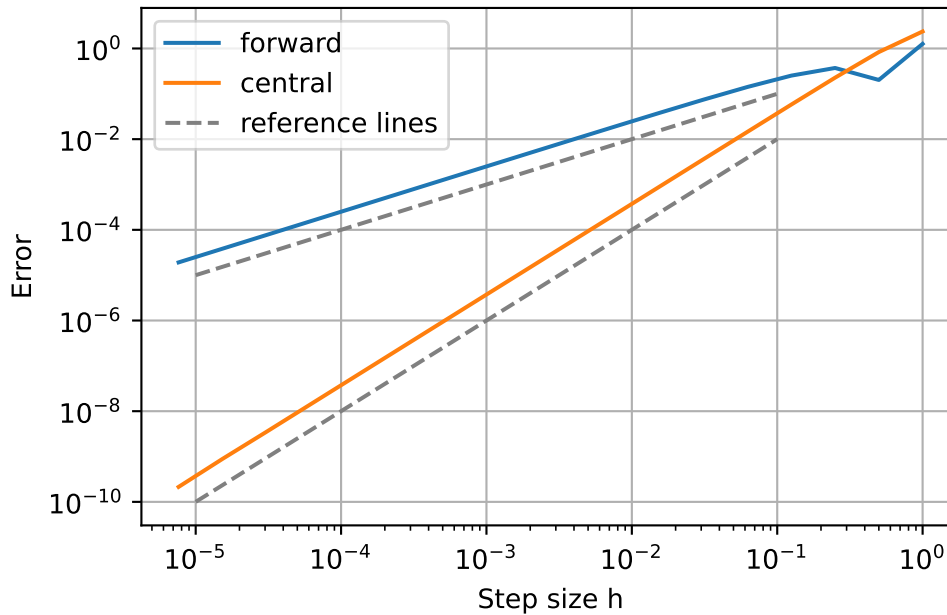
```
x0 = 0.85  
  
hs = []  
fpfs = []  
fpcs = []  
  
h0 = 1  
for i in range(18):  
    h = h0 / 2**i  
  
    f0 = func(x0)  
    fnh = func(x0 - h)  
    fph = func(x0 + h)  
  
    fpf = (fph - f0) / h  
    fpc = (fph - fnh) / (2*h)  
  
    hs.append(h)  
    fpfs.append(fpf)  
    fpcs.append(fpc)
```

```
plt.plot(hs, np.abs(fpfs - fp_exact), label='forward')  
plt.plot(hs, np.abs(fpcs - fp_exact), label='central')  
  
plt.plot([1e-5, 1e-1], [1e-5, 1e-1], '--', color='grey', label='reference lines')  
plt.plot([1e-5, 1e-1], [1e-10, 1e-2], '--', color='grey')
```

```
plt.xlabel('Step size h')
plt.ylabel('Error')

plt.xscale('log')
plt.yscale('log')

plt.legend()
plt.grid();
```



In the logarithmic representation of both axes, power functions become straight lines with the exponent as the slope. This means that the error in the above plot behaves like a power function of degree one and two, respectively. The reference lines that are drawn have slopes of one and two, respectively. This also corresponds to the order $\mathcal{O}(h)$ and $\mathcal{O}(h^2)$ from the difference formula.

3.4.2 Rounding Errors

If the step size is reduced even further, the precision of number representation, i.e., rounding errors, begins to affect the approximation.

```
x0 = 0.85
hs = []
```

```

fpfs = []
fpcs = []

h0 = 1
for i in range(35):
    h = h0 / 2**i

    f0 = func(x0)
    fnh = func(x0 - h)
    fph = func(x0 + h)

    fpf = (fph - f0) / h
    fpc = (fph - fnh) / (2*h)

    hs.append(h)
    fpfs.append(fpf)
    fpcs.append(fpc)

plt.plot(hs, np.abs(fpfs - fp_exact), label='forward')
plt.plot(hs, np.abs(fpcs - fp_exact), label='central')

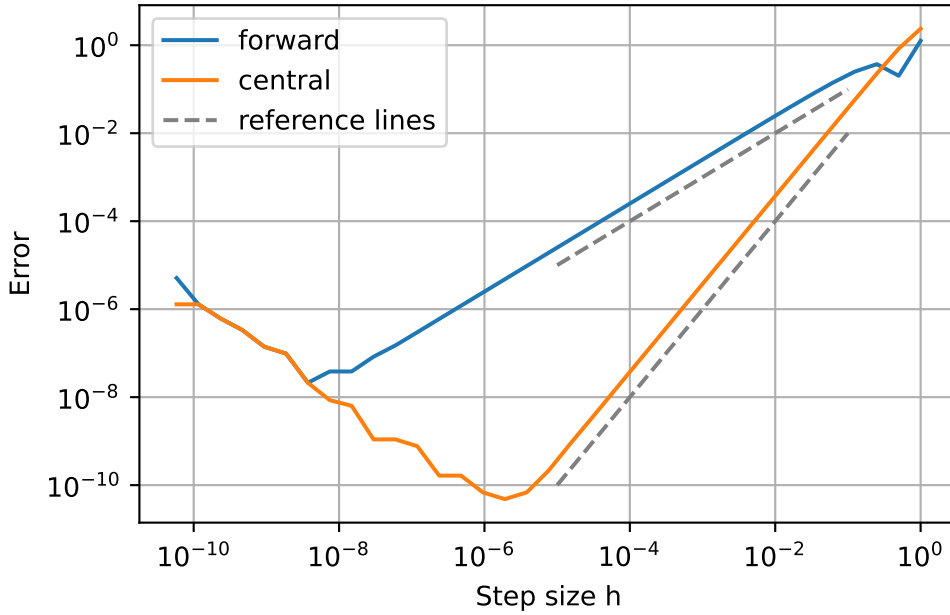
plt.plot([1e-5, 1e-1], [1e-5, 1e-1], '--', color='grey', label='reference lines')
plt.plot([1e-5, 1e-1], [1e-10, 1e-2], '--', color='grey')

plt.xlabel('Step size h')
plt.ylabel('Error')

plt.xscale('log')
plt.yscale('log')

plt.legend()
plt.grid();

```



As previously introduced, 64-bit numbers can only be represented with a precision of approximately $\approx 10^{-16}$. This means that, for example, the difference between two numbers cannot be computed more accurately than . This is the so-called rounding error.

In the specific case of the forward difference formula, this implies:

$$\begin{aligned}
 f'(x_0) &= \frac{f(x_0 + h) - f(x_0)}{h} + \mathcal{O}(h) \\
 \stackrel{\text{rounding error}}{\Rightarrow} & \frac{f(x_0 + h) - f(x_0) + \mathcal{O}(\epsilon)}{h} + \mathcal{O}(h) \\
 &= \frac{f(x_0 + h) - f(x_0)}{h} + \mathcal{O}\left(\frac{\epsilon}{h}\right) + \mathcal{O}(h)
 \end{aligned}$$

Hence, reducing h only makes sense as long as the rounding error is small compared to h . More precisely:

$$\begin{aligned}
 \bar{h} &\leq h \\
 \Rightarrow h &\geq \sqrt{\epsilon}
 \end{aligned}$$

With $\epsilon \approx 10^{-16}$, for this difference formula, a h is only reasonable down to about 10^{-8} .